

POC - Evidence Documentation

UPLOADING THE DOCUMENTS IN AI SYSTEM

The screenshot displays a web application titled "AI-Driven Knowledge Retrieval for Tax Platform Modernization". The interface is divided into two main sections: "Sources" on the left and "Chat" on the right.

Sources Section:

- A header bar includes a "+ Add sources" button.
- A search bar prompts the user to "Search the web for new sources" with dropdown menus for "Web" and "Fast Research", and a search icon.
- A list of document sources is shown, each with a checkbox for selection:
 - Application-Overview.md
 - Architecture.md
 - Business-Problem.md
 - Data-Lineage.md
 - Incident-History.md
 - Program-Inventory.md
 - Vendor-Documentation.md
- A "Select all" checkbox is located at the top right of the list.

Chat Section:

- A header bar includes a "Customize" button.
- A large header card displays the title "AI-Driven Knowledge Retrieval for Tax Platform Modernization" and indicates "7 sources · Jun 16, 2026".
- The main content area contains a paragraph of text:

These sources detail a technical proposal and proof-of-concept for an **AI-driven knowledge discovery platform** designed to navigate a complex, legacy tax processing environment. The system utilizes **Retrieval-Augmented Generation (RAG)** to consolidate fragmented technical documentation into a centralized, searchable repository for stakeholders like architects and support teams. By moving away from a heavy **reliance on subject matter experts**, the architecture allows users to perform **natural language queries** to identify business rules, analyze data lineage, and assess the impact of legislative changes. This approach aims to **mitigate operational risks** and streamline modernization efforts by providing accurate, source-grounded insights into COBOL-based systems. Ultimately, the project seeks to transform **institutional knowledge** into a strategic asset that improves decision-making and reduces the time required for technical investigations.
- A footer bar contains a text input field "Ask a question or create something", a "7 sources" indicator, and a send button.

BUSINESS RULE DISCOVERY

Which documents define penalty calculation logic and where is it implemented?

Chat

Which documents define penalty calculation logic and where is it implemented?

Penalty calculation logic within the Tax Processing Platform is defined across multiple documentation sources and implemented through specific COBOL programs and configuration tables.

Documents Defining Penalty Logic

According to the sources, the following documents provide the definitions and rules for penalty calculations:

- **Vendor-Documentation.md:** This is a primary source that outlines the functional **Penalty Assessment Rules** 1 - 3 . It specifies that penalties are triggered by events such as late filing, late payment, under-reporting of liabilities, and other non-compliance events 2 .
- **Business-Problem.md:** This document identifies penalty assessment logic as a critical business rule that is frequently embedded within both vendor documentation and COBOL source code 4 .
- **Application-Overview.md:** It classifies penalty and interest processing as a major functional domain and a core part of the system's mission-critical revenue administration 5 6 .

Implementation Details

The technical implementation of these rules is distributed across the following components:

- **COBOL Program TXN520:** This is the primary module responsible for **Penalty Assessment** 7 8 . Its function is to apply penalties for non-compliance and it generates `PENALTY_TRANSACTION` records 7 9 .
- **Configurable Control Tables:** While the logic is in the code, the actual penalty rates are maintained within **control tables** 2 . This allows for rule updates without necessarily changing the underlying program code, though such changes still require significant impact analysis 10 11 .
- **Batch Processing Stream:** Penalty assessment is executed as a high-volume activity

Ask a question or create something



What are the business rules governing statutory interest calculation, and in which COBOL programs or vendor documents are these rules implemented?

What are the business rules governing statutory interest calculation, and in which COBOL programs or vendor documents are these rules implemented?

Statutory interest calculation is a core financial function of the platform, governed by specific business rules and technical implementations:

Business Rules for Interest Calculation

- **Calculation Frequency:** Interest is calculated **daily** on all eligible **outstanding taxpayer balances** ¹ ² .
- **Key Factors:** The calculation is based on the **outstanding liability amount**, the **applicable interest rate**, the **number of overdue days**, and specific **regulatory calculation rules** ² .
- **Rate Management:** Interest rates are not fixed but are maintained within **configurable interest rate control tables** to accommodate legislative updates ² .
- **Processing Sequence:** In the nightly batch cycle, interest calculation is **Step 5**, occurring after Penalty Assessment and before Refund Processing ³ .
- **Impact Requirements:** Any changes to these rates require a mandatory **impact assessment** across reporting, payment, refund, and compliance components ² .

Implementation Details

- **COBOL Program TXI510:** This is the primary module for **Interest Calculation** ⁴ ⁵ . It calculates interest on overdue liabilities and generates **INTEREST_TRANSACTION** records ⁵ .
 - **Vendor Documentation:** The functional logic and rules are defined in the **Vendor-Documentation.md** under the "Penalty and Interest Processing" and "Interest Calculation Rules" sections ¹
-
- **Supporting Documentation:** The "Penalty and interest calculation rules" are also identified as critical rules in **Business-Problem.md** ⁷ .
 - **Dependencies:** The implementation in **TXI510** depends on **Liability Calculation** data and the **Interest Rate Control Tables** ⁵ .

APPLICATION DISCOVERY – UNDERSTANDING THE SYSTEM

Which COBOL programs, batch jobs, and system components are involved in refund processing within the Tax Processing Platform?

Which COBOL programs, batch jobs, and system components are involved in refund processing within the Tax Processing Platform?

Refund processing in the Tax Processing Platform involves a specific set of COBOL programs, batch jobs, and technical components that operate within a strictly controlled sequence.

COBOL Programs

- **TXR610 (Refund Processing):** The **primary program** that determines eligibility, performs fraud and compliance screening, and initiates approved refund transactions [1](#) [2](#) .
- **TXL320 (Liability Calculation):** An **upstream dependency** that calculates the liabilities required to determine if an overpayment exists [1](#) [3](#) .
- **TXP410 (Payment Reconciliation):** An **upstream dependency** that reconciles payments against liabilities; refunds are often triggered when this program identifies excess credits [1](#) [4](#) .
- **TXC710 (Compliance Management):** A **downstream consumer** that investigates refund anomalies and exceptions [5](#) [6](#) .
- **TXS900 (Statutory Reporting):** A **downstream consumer** that uses refund data for regulatory submissions [7](#) [8](#) .
- **TXA810 (Audit Reporting):** A **downstream consumer** that captures refund transactions in the system-generated audit history [7](#) .

Batch Jobs

- **Nightly Processing Stream (Step 6):** Refund processing is executed as the **sixth step** in the nightly batch cycle [9](#) .
- **Sequence Dependencies:** It must follow the successful completion of Return Validation, Liability Calculation, Payment Reconciliation, Penalty Assessment, and Interest Calculation [9](#) .
- **High-Volume Processing:** It is one of the approximately 400 scheduled jobs designed to handle high-volume business activities [10](#) [11](#) .

System Components

- **Refund Payment Interface:** This external interface receives approved `REFUND_TRANSACTION` records from the platform to facilitate actual payment generation [2](#) [...](#) .
- **DB2 Databases:** Used to store and retrieve core data elements such as `REFUND_REQUEST` and `REFUND_TRANSACTION` records [1](#) [13](#) .
- **Control Tables:** These modules contain the configurable rules used for eligibility determination; failure to update these can lead to batch processing failures [13](#) [...](#) .
- **Reporting Components:** Specialized components that generate **Refund Activity Reports** for management and operational oversight [13](#) [16](#) .

IMPACT ANALYSIS

What is the impact of modifying penalty calculation rules?

What is the impact of modifying penalty calculation rules?

Modifying penalty calculation rules within the Tax Processing Platform (TPP) has a significant impact because these rules are deeply embedded in the system's core financial logic and nightly processing sequence 1 2 3 .

The impacts of such a change include:

1. Direct Technical and Programmatic Impact

- **COBOL Program TXN520:** This is the primary module for **Penalty Assessment** and would require direct modification or updated configuration 4 5 .
- **Downstream Reporting:** Any change to penalty logic directly impacts **Statutory Reporting (TXS900)** and **Audit Reporting (TXA810)**, as these modules depend on the penalty data generated by TXN520 6 .
- **Account Maintenance:** Changes affect **Taxpayer Account Maintenance**, where penalty transactions are recorded as separate financial events against taxpayer balances 7 8 .

2. Data and Lineage Consequences

- **Transaction Records:** Modifying the rules changes the generation of **PENALTY_TRANSACTION** records, which are critical data elements consumed by taxpayer accounts and regulatory bodies 9 10 .
- **Interdependent Calculations:** Because penalty assessment depends on **Interest Calculation (TXI510)** and compliance rules, changes must be synchronized to ensure financial accuracy 4 7 .

3. Operational and Delivery Risks

- **Data Inconsistency:** Historical incident records show that a "penalty calculation change" previously introduced data inconsistencies that caused **Critical** failures in monthly regulatory reporting (Incident TPP-2025-142) 11 .
- **Broad Systemic Reach:** A seemingly simple change to these rules can impact multiple

What is the impact on COBOL programs, batch jobs, and downstream reports if the statutory interest rate logic is modified in the Tax Processing Platform?

What is the impact on COBOL programs, batch jobs, and downstream reports if the statutory interest rate logic is modified in the Tax Processing Platform?

Modifying the statutory interest rate logic in the Tax Processing Platform (TPP) has a broad, systemic impact because interest calculation is a core financial function tightly integrated into the nightly batch sequence and downstream reporting [1](#) [2](#) .

Impact on COBOL Programs

The modification directly affects the primary calculation engine and several dependent modules:

- **TXI510 (Interest Calculation):** This is the central program responsible for calculating daily statutory interest on overdue liabilities [3](#) . It must be updated to reflect new logic or to correctly consume updated parameters from the **Interest Rate Control Tables** [3](#) .
- **TXN520 (Penalty Assessment):** As a direct downstream consumer of interest data, this program is impacted because penalty assessments often depend on the results of interest calculations [3](#) [4](#) .
- **TXR610 (Refund Processing):** While not a direct consumer of the interest calculation program, refund processing is listed as a potentially affected component because interest adjustments can alter the final credit or debit balance used to determine refund eligibility [1](#) [5](#) .
- **Reporting Programs: TXS900 (Statutory Reporting) and TXA810 (Audit Reporting)** are impacted as they must consume and accurately represent the new interest transaction data in regulatory submissions and audit logs [3](#) [...](#) .

Impact on Batch Jobs

Interest calculation is a critical step in the TPP's strictly controlled nightly batch stream:

- **Processing Sequence:** It is executed as **Step 5** in the nightly cycle, following Penalty Assessment (Step 4) and preceding Refund Processing (Step 6) [8](#) . Any logic change requires careful validation to ensure it does not cause an **abend** (abnormal end) that would halt the entire downstream stream, such as regulatory reporting [8](#) [9](#) .
- **Reconciliation and Balancing:** Changes to interest logic impact the daily financial reconciliation activities performed during the batch window to ensure taxpayer accounts remain accurate [5](#) [10](#) .
- **High-Volume Processing:** Since interest is calculated daily for all eligible outstanding balances, the updated job must be tuned to avoid resource issues like **DB2 table lock contention**, which has historically caused batch failures during financial processing [9](#) [11](#) .

Impact on Downstream Reports

The TPP's reporting layer is highly dependent on the successful and accurate completion of upstream interest activities [12](#) . Impacted reports include:

- **Statutory and Regulatory Reports:** Program **TXS900** uses interest data to generate mandatory government submissions; any inconsistency in the new logic could lead to validation failures or missed regulatory deadlines [6](#) [...](#) .
- **Audit History and Logs:** Every interest transaction is recorded as a system-generated activity with a timestamp [12](#) . Modifying the logic requires ensuring that the **Audit Reporting (TXA810)** accurately captures the "why" and "when" of the new calculations for regulatory compliance [6](#) [14](#) .

- **Financial Reconciliation Reports:** These reports compare processed transactions against account balances and will reflect any shifts in liability caused by the interest rate change 5 15 .
- **Management and Operational Reports:** This includes daily activity reports and refund activity reports, which may show different totals based on the updated financial outcomes 12 15 .

Required Testing and Validation

Due to the complexity of these dependencies, a modification to interest logic requires extensive regression testing of:

- **Interest and penalty calculations** to ensure mathematical accuracy 5 .
- **Financial reconciliation** to confirm taxpayer account balances remain consistent 5 .
- **Reporting validation** to prevent data inconsistencies that have historically caused critical failures in monthly regulatory reporting (e.g., Incident TPP-2025-142) 5 13 .

DATA LINEAGE

Trace the end-to-end lifecycle of taxpayer refund amount from creation in COBOL programs to its final consumption in reporting systems.

Trace the end-to-end lifecycle of taxpayer refund amount from creation in COBOL programs to its final consumption in reporting systems.

The end-to-end lifecycle of a taxpayer refund within the Tax Processing Platform (TPP) is a strictly ordered process that transitions from financial reconciliation to transaction generation and final regulatory reporting.

1. Prerequisites and Initiation

Refund processing is **Step 6** in the nightly batch sequence and only begins after the successful completion of upstream financial calculations **1** **2**.

- **Upstream Inputs:** The process depends on **TXL320** (Liability Calculation) and **TXP410** (Payment Reconciliation) to determine the taxpayer's final balance **3** **4**.
- **Triggers:** A refund is initiated when payments exceed liabilities, credits exist on the account, or regulatory adjustments create overpayments **4** **5**.

2. Eligibility and Validation (TXR610)

The COBOL program **TXR610** is the core engine for refund administration **3**.

- **Determination:** It evaluates the account to verify eligibility and performs automated **fraud screening** and **compliance validation** **3** **5**.
- **High-Value Review:** Any refund request exceeding predefined financial thresholds is routed for a manual **compliance review** before approval **5**.
- **Compliance Integration:** Data from this stage is also consumed by **TXC710** (Compliance Management) to identify anomalies **6**.

3. Transaction Creation



Once approved, the system moves from analysis to formal financial recording.

- **Data Generation:** The program generates a **REFUND_TRANSACTION** record ³ ⁷ .
- **Account Update:** This transaction is posted to the taxpayer's consolidated account, specifically updating the **refund balances** ⁸ .

4. Payment Distribution

The platform must then bridge the gap between tax accounting and actual payment issuance.

- **Interface Transfer:** The **REFUND_TRANSACTION** is sent to the **Refund Payment Interface** ⁶ ⁷ .
- **External Execution:** This interface transfers the data to downstream systems responsible for physical payment generation ⁵ .

5. Final Consumption in Reporting Systems

The lifecycle concludes when the refund data is consumed by reporting and audit modules to satisfy regulatory requirements ⁹ ¹⁰ .

- **Statutory Reporting (TXS900):** This program consumes refund transaction data to generate mandatory **regulatory reporting submissions** for government agencies ¹¹ ¹² .
- **Audit Reporting (TXA810):** The refund is captured in the **Audit History and Transaction Log**, including a timestamp and system-generated activity log to ensure the process is fully auditable ¹⁰ ¹¹ .
- **Management Oversight:** The data is used to produce **Refund Activity Reports** for daily operational monitoring ¹⁰ .

PRODUCTION SUPPORT

A refund batch job fails midway with an ABEND error. Based on incident history and program inventory, what are the possible root causes, impacted components, and recovery steps?

A refund batch job fails midway with an ABEND error. Based on incident history and program inventory, what are the possible root causes, impacted components, and recovery steps?

An ABEND (abnormal end) error in a refund batch job within the Tax Processing Platform (TPP) is a high-priority event that disrupts the strictly controlled nightly processing sequence [1](#) [2](#) . Based on the program inventory and historical incident data, the details of such a failure are as follows:

Possible Root Causes

- **Configuration and Release Failures:** A primary cause for refund stream failures is a **lack of configuration promotion**. For example, failing to promote a control table update during a release implementation has previously caused the refund eligibility validation to fail (Incident TPP-2025-041) [1](#) .
- **Database Resource Conflicts:** High-volume batch processing is susceptible to **DB2 table lock contention**, which can cause update failures when the system attempts to record transactions or update account balances (Incident TPP-2025-087) [3](#) [4](#) .
- **Upstream Dependency Issues:** Refund processing (**TXR610**) depends on the successful completion of **Payment Reconciliation (TXP410)** and **Liability Calculation (TXL320)** [2](#) [5](#) . If these upstream programs provide incomplete or inconsistent data, the downstream refund job may fail during validation [4](#) [6](#) .
- **Data Quality Problems:** Unexpected data quality issues or a sudden surge in exception volumes following regulatory changes can exceed operational thresholds and trigger processing failures [4](#) [7](#) .



Impacted Components

- **Primary Program (TXR610):** As the module responsible for determining eligibility and initiating transactions, this is the immediate point of failure 5.
- **Data Generation:** The creation of **REFUND_TRANSACTION** records is delayed, which directly impacts the accuracy of taxpayer account balances 1 8.
- **Downstream Applications:**
 - **Refund Payment Interface:** This external system will not receive the approved transactions necessary to issue payments 9.
 - **Statutory Reporting (TXS900):** This module will have incomplete data, leading to failed validation checks and risks to regulatory deadlines 1 10.
 - **Compliance Management (TXC710):** Since this program depends on refund processing data, investigation queues and exception processing may be delayed 9.
- **Operational Impact:** Observed effects include **delayed refund generation**, incomplete regulatory reporting, and an **increase in customer service inquiries** 1.

Recovery Steps

1. **Identify and Resolve Root Cause:**
 - If the failure was due to missing configurations, the **missing control table updates must be promoted** to the production environment 1.
 - If caused by database contention, technical teams must **resolve the lock contention** 11.
 - If upstream data was at fault, the **correct data files must be received** and the prerequisite reconciliation or liability steps must be rerun 12.
2. **Restart the Affected Job:** Once the cause is addressed, the refund job (TXR610) should be **restarted from the failed step** or from a designated checkpoint 1 11.
3. **Validate the Batch Sequence:** Because the TPP follows a **controlled nightly sequence**, support teams must ensure all upstream steps (Steps 1–5) are successfully completed before the refund process (Step 6) is finalized 2.
4. **Regenerate Reporting:** After a successful restart and completion of the refund job, any **downstream reporting (TXS900)** must be regenerated to ensure data consistency and to meet regulatory obligations 6.

MODERNIZATION RISK

Which components of the Tax Processing Platform are high-risk for modernization, and what makes them complex or tightly coupled?

Which components of the Tax Processing Platform are high-risk for modernization, and what makes them complex or tightly coupled?

Based on the provided documents, the high-risk components for modernization within the Tax Processing Platform are primarily those that manage core financial logic and exhibit extensive dependency relationships.

High-Risk Components for Modernization

The **Program Inventory** explicitly identifies the following four COBOL programs as having high business criticality and extensive dependencies, making them the most sensitive candidates for modernization ¹ :

- **TXL320 – Liability Calculation:** Calculates taxpayer liabilities based on return info and tax rules ¹ ² .
- **TXP410 – Payment Reconciliation:** Applies payments against outstanding liabilities ¹ ³ .
- **TXI510 – Interest Calculation:** Calculates daily statutory interest on overdue balances ¹ ⁴ .
- **TXN520 – Penalty Assessment:** Applies financial penalties for non-compliance events ¹ ⁴ .

Reasons for Complexity and Tight Coupling

The complexity and risks associated with these components stem from several structural and functional factors:

- **Embedded Business Rules:** Many critical processing rules are deeply embedded within the **COBOL source code** itself, rather than being managed in externalized, modern business rule engines ⁵ ⁶ .
- **Interconnected Data Lineage:** The platform features highly interconnected data flows ⁷ . For example, **TAX_LIABILITY** data generated by TXL320 is a required input for nearly every other core process, including Payment Processing, Interest Calculation, Penalty Assessment, Refund Processing, and Reporting ⁸ .
- **Strict Batch Sequencing:** The platform operates in a **strictly controlled nightly sequence** ⁹ . Because each step depends on the successful output of the previous one (e.g., Interest depends on Penalty, which depends on Liability), a change or failure in one high-risk module has immediate cascading effects on all downstream activities ⁹ ¹⁰ .
- **Distributed Knowledge:** Essential business logic and technical knowledge are fragmented across vendor manuals, legacy code, and incident records ¹¹ ¹² . This makes it difficult to perform a complete impact analysis without significant manual effort and dependency on senior subject matter experts ¹³ ¹⁴ .
- **Multi-Layered Impact:** A seemingly minor change to a core component (like statutory interest) can impact multiple technical layers simultaneously, including batch jobs, online transactions, database structures, external interfaces, and regulatory reports ¹³ ¹⁵ .

FULL SYSTEM SYNTHESIS

Explain the Tax Processing Platform end-to-end, covering business functions, technical architecture, operational processes, and data flow across the system.

Explain the Tax Processing Platform end-to-end, covering business functions, technical architecture, operational processes, and data flow across the system.

The **Tax Processing Platform (TPP)** is a mission-critical government revenue system responsible for the end-to-end administration of taxpayer services, including return processing, liability calculation, and revenue collection 1 2 .

1. Core Business Functions

The platform supports several primary functional domains 3 4 :

- **Taxpayer Registration:** Creation and maintenance of consolidated taxpayer master records 3 5 .
- **Return Processing:** Validation of submitted tax returns for arithmetic accuracy, duplicates, and compliance 3 6 .
- **Payment & Refund Processing:** Collection and reconciliation of payments against liabilities, and the assessment and issuance of refunds when credits exist 3
- **Penalty and Interest Processing:** Daily calculation of statutory interest on overdue balances and application of penalties for non-compliance (e.g., late filing) 4
- **Compliance Management:** Identification and investigation of anomalies, such as refund exceptions or missing returns 4 10 .
- **Reporting:** Generation of operational, management, and mandatory regulatory reporting 4 10 .

2. Technical Architecture

The TPP is a large-scale legacy environment characterized by:



- **Technology Stack:** Over **700 vendor-supplied COBOL programs** and **400 scheduled batch jobs** running against **DB2 databases** 11 12 .
- **Workload Types:** A dual-workload model featuring **online services** for real-time account maintenance and inquiries, and **batch processing** for high-volume automated tasks 3
- **Embedded Logic:** Critical tax rules are often embedded directly within COBOL source code and configurable vendor control tables 6 14 .
- **Knowledge Retrieval:** The architecture uses **Retrieval-Augmented Generation (RAG)** to correlate distributed information across vendor manuals, incident records, and program inventories for decision support 15 16 .

3. Operational Processes

Operations are governed by a strictly controlled **nightly batch sequence** where the failure of one step typically halts downstream activities 17 :

1. **Return Validation (TXR210)**
2. **Liability Calculation (TXL320)**
3. **Payment Reconciliation (TXP410)**
4. **Penalty Assessment (TXN520)**
5. **Interest Calculation (TXI510)**
6. **Refund Processing (TXR610)**
7. **Regulatory Reporting (TXS900)**

The support model is split between an **external vendor** (responsible for maintenance and development) and **internal teams** (responsible for production support, incident management,

and release implementation) 4 18 .

4. End-to-End Data Flow

Data flows through the platform in a highly interconnected lineage 19 20 :

- **Initiation:** A **TAXPAYER_ID** is assigned during registration and is consumed by all platform components 21 .
- **Submission:** **TAX_RETURN** data is submitted and validated, feeding into **TAX_LIABILITY** calculations 21 22 .
- **Reconciliation:** **PAYMENT_TRANSACTION** records from external systems are reconciled against liabilities 22 .
- **Financial Assessment:** Outstanding balances trigger **INTEREST_TRANSACTION** and **PENALTY_TRANSACTION** records 19 23 .
- **Closing:** Approved overpayments result in **REFUND_TRANSACTION** records, which are sent to external payment interfaces and finally consumed by **Statutory and Audit Reporting** 19 23 .

GOVERNANCE CONFLICT

If there is a conflict between vendor documentation and incident history regarding system behavior, which source should be trusted for production decision-making and why?

If there is a conflict between vendor documentation and incident history regarding system behavior, which source should be trusted for production decision-making and why?

In a production conflict, **Incident History** is generally the more trusted source for understanding **actual system behavior**, while **Vendor Documentation** represents the **intended functional design** ¹

The reasons for this prioritization include:

- **Observed Reality vs. Theory:** Incident records provide "valuable operational knowledge" based on real-world outcomes, whereas vendor documentation provides a high-level summary of how the platform is *supposed* to work ²
- **Operational Reliability:** Support teams specifically rely on incident history and recovery procedures to handle production events, as these records capture nuances—like the DB2 table lock contention in Incident TPP-2025-087—that documentation may not detail ¹ ⁵ .
- **Implementation Gaps:** Incident TPP-2025-041 demonstrates that production behavior can deviate from documentation due to configuration errors or missed updates, making the record of the failure a more accurate reflection of the current environment ⁶ .
- **Decision Support:** While documentation is used for business rule discovery, incident history is critical for "operational decision support" during high-priority investigations ⁷ ⁸ .

Important Note: The sources emphasize that any information retrieved from these records should be treated as **decision-support information** rather than authoritative guidance and must always undergo **human validation** by experienced personnel ⁸